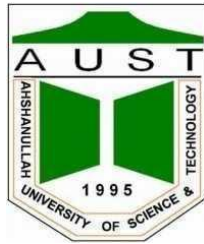


Ahsanullah University of Science & Technology

Department of Computer Science & Engineering



Final Project Report

Houses from Outside

Project Serial No. 08

Computer Graphics Lab (CSE 4204)

Submitted By:

Name	Student ID
Jakaria Mahmud	20220104141
Md. Redwan Hossen	20220104147
Section: C (C2) Group No: 07	
Date of Submission: 30 September 2026	

1. Problem Analysis

1.1 Project Requirements

The assigned project is Serial No. 08, titled “Houses from outside”. The assignment sheet defines its scope through four columns:

- **3D Objects:** (1) a building [with texture], (2) a cottage [with texture].
- **Keyboard interaction:** the camera will move around the houses.
- **Mouse interaction:** the texture of the cottage will change.
- **Animation:** the light position will rotate around the building.

In addition, the instructions tab lists six requirements that are mandatory for every group regardless of the assigned topic: custom shaders, implementation of lighting, perspective projection, texture for each object, animation, and mouse and keyboard interaction. The prescribed tool is Three.js and the project carries 25 marks.

From these constraints the following concrete goals were derived. Two clearly distinguishable textured houses must dominate the scene: a tall multi-storey building and a small gabled cottage. The camera must be driven entirely from the keyboard and must be able to travel around both houses rather than merely zoom. A mouse click on the cottage, and only on the cottage, must visibly change its wall texture. A light source must physically orbit the building so that both the moving highlight and the moving shadow are obvious to an observer. Finally, every surface in the scene must carry a texture, and a significant part of the rendering must use shaders written by us rather than Three.js built-in materials.

1.2 Core Integration

The project is written as a set of native ES modules with no build step, no bundler and no npm dependency tree; the only third-party file is the Three.js library itself. The application is split into twelve focused modules so that each mandatory requirement maps onto a small, readable file:

- `three-loader.js` — loads Three.js from the local vendor folder, falling back to two CDNs.
- `config.js` — every tunable constant: camera, plot layout, lights, sky palette, texture skins, camera presets.
- `shaders.js` — all hand-written GLSL vertex and fragment programs.
- `materials.js` — factory functions that assemble the `THREE.ShaderMaterial` objects.
- `textures.js` — texture loading, sRGB tagging, anisotropic filtering and tiling.
- `lighting.js` — the lights, the shadow maps, the day-night cycle and the orbiting light.
- `building.js` and `cottage.js` — the two required models.
- `environment.js` — ground, sky dome, road, trees, street lamp and props.
- `controls.js` — the custom keyboard and mouse camera rig.
- `hud.js` and `main.js` — the on-screen panel, the renderer, the raycasting and the render loop.

The first integration point between these modules is a single set of shared light uniform objects created in `lighting.js`. Every custom shader material spreads the same uniform

objects into its own uniform map, so when the day-night cycle writes one value the change is picked up by the sky, the facade, the cottage walls and all foliage within the same frame, with no per-object book-keeping.

The second integration point is the texture library. `textures.js` loads the 31 generated PNG files once, tags the colour maps as sRGB, applies the maximum anisotropy the GPU reports, and then hands out either raw tiled textures for the custom shaders or fully configured `MeshStandardMaterial` instances for the smaller props. This is what allows custom-shader surfaces and built-in-material props to stand next to each other without a visible seam.

The models themselves are built entirely from Three.js primitive geometries (`BoxGeometry`, `CylinderGeometry`, `PlaneGeometry`, `TorusGeometry` and `Points`), grouped into a `THREE.Group` per structure so the whole house can be positioned and rotated as one object.

1.3 Technical Challenges

Camera transformation. The camera is held on a spherical coordinate system around a movable target T . For a radius r , azimuth θ and elevation φ the eye position is computed as $x = T_x + r \sin \varphi \sin \theta$, $y = T_y + r \cos \varphi$, $z = T_z + r \sin \varphi \cos \theta$, with φ clamped to (0.14, 1.52) radians so the camera can never flip over the pole or sink under the ground plane. Every key press changes a target value rather than the value itself, and each frame the actual value is moved towards it by a factor of $1 - e^{-k \Delta t}$. This exponential damping is frame-rate independent, so the motion feels identical at 30 fps and at 144 fps.

Animation of the light. The required animation is a light that rotates around the building. Its position is a simple parametric circle evaluated every frame: $x = c_x + R \cos(\omega t)$, $z = c_z + R \sin(\omega t)$, $y = h$, where c is the centre of the building, $R = 17.5$, $h = 13.5$ and $\omega = 0.42$ rad/s. The same position is written into the `PointLight`, into the glowing orb mesh and into the `uOrbitPos` uniform shared by every custom shader, which keeps the real light, the visible orb and the shaded highlight perfectly in step.

Matching custom shaders to Three.js lighting units. Our fragment shaders implement Blinn-Phong shading by hand, while the smaller props use `MeshStandardMaterial`, which is physically based. A directional light of intensity 2.15 does not produce the same brightness in both models, because Three.js PBR materials divide the incoming radiance by π internally. We therefore drive the shader uniforms from the same light objects but divide by π , and we calibrated separate gains for the two point lights at a reference distance of about twelve units.

Colour management. Three.js applies output colour-space conversion only to its own materials; a raw `ShaderMaterial` writes whatever it puts into `gl_FragColor`. To keep the two consistent we disabled tone mapping, decoded every texture sample with a hand-written `sRGBToLinear` function, performed all lighting maths in linear space, and encoded the result with `linearToSRGB` before writing it out. Linear fog had to be reimplemented inside each shader as well, because `scene.fog` is only injected into built-in materials.

Normals under transformation. The shaders compute world-space normals as `mat3(modelMatrix) * normal`, which is only correct when the object is not non-uniformly scaled. Instead of computing a full inverse-transpose normal matrix on the GPU we adopted a discipline in the model code: every box is created at its true size with `new THREE.BoxGeometry(w, h, d)` and is never scaled, and foliage is only ever scaled uniformly.

Selective picking. The raycaster must react to the cottage only, so an explicit `clickTargets` array holds just the cottage wall and gable meshes. A mouse release is ignored if the pointer moved more than a few pixels since the press, which prevents an orbit drag from accidentally changing the texture.

Running without internet. The demonstration machine may have no network access, so `three-loader.js` tries three sources in order — the local vendor folder, then unpkg, then jsDelivr — and prints a readable instruction panel if all three fail. A copy of the library is also kept inside `vendor`, so the demonstration never depends on the network.

Textures without copyright risk. Rather than downloading texture images we wrote `tools/gen_textures.py`, which procedurally generates all 31 PNG files, including normal maps derived from the height field of each pattern, using Pillow and NumPy. Every pixel in the project is therefore our own work.

2. User Interaction

2.1 Key Interaction

The keyboard requirement for this project is that the camera will move around the houses. Rather than importing `OrbitControls` we implemented the camera rig ourselves in `controls.js`, so the interaction is genuinely our own code. A key press does not move the camera directly; it changes a target radius, azimuth, elevation or look-at point, and the render loop eases the current values towards those targets with exponential damping.

Camera movement:

- `W / S` — dolly the camera towards or away from the houses.
- `A / D` — orbit left or right around the houses.
- `Q / E` — decrease or increase the elevation angle.
- `R / F` — raise or lower the look-at point.
- Arrow keys — pan the look-at point across the plot.
- `Shift` — hold to move 2.4 times faster.
- `1` to `5` — jump to a camera preset with eased interpolation: front view, cottage close-up, building close-up, aerial overview and street level.
- `[` and `]` — narrow or widen the field of view between 25° and 95° .
- `V` — reset the view and the field of view. `Space` — automatic orbit tour.

Scene control from the keyboard:

- `C` — advance the cottage to the next texture (the keyboard equivalent of clicking it).
- `B` — toggle the building facade between red brick and grey concrete.
- `T` — pause or resume the day-night cycle. `Y` — skip time forward by about three hours.
- `L` — pause or resume the orbiting light. `O` — show or hide its orbit path ring.
- `K` — switch the foliage between summer and autumn. `M` — cycle the wind strength.
- `X` — turn the shadow maps on or off. `P` — save a PNG snapshot of the current frame.
- `H` — hide or show the HUD. `G` — open or close the help overlay.

All key handling ignores events carrying `Ctrl`, `Alt` or `Meta` so that ordinary browser shortcuts keep working, and the five camera presets make the demonstration reproducible.

2.2 Mouse Interaction

The mouse requirement for this project is that the texture of the cottage will change, and it is implemented with `THREE.Raycaster` picking in `main.js`.

On every pointer move the normalised device coordinates of the cursor are recorded. Several times per second a ray is cast from the camera through that point and tested against the cottage `clickTargets` array. When the ray hits the cottage the `uHighlight` uniform of the wall shader is raised, which adds a blue Fresnel rim around the silhouette, the cursor changes to a pointer and a tooltip appears showing the name of the current texture. The building and the environment are deliberately excluded from the pick list.

When the left button is released over the cottage, and the pointer did not move far enough during the press to count as a camera drag, the cottage advances to the next of its four materials: Wooden Planks, Stone Masonry, White Plaster and Beige Brick, then back to the beginning.

The change is not a hard texture swap. The wall shader holds two colour maps and two normal maps at the same time, and a `uMix` uniform is animated from 0 to 1 over roughly 0.45 seconds, cross-fading both the albedo and the surface normals so the switch looks like a genuine change of material. When the fade completes the incoming pair becomes the base pair, ready for the next click. A toast message confirms the new material and the HUD updates its cottage texture row.

The remaining mouse bindings support the camera: left drag orbits, right or middle drag pans the target, and the mouse wheel zooms in and out.

3. Design and Aesthetic Aspects

3.1 Visual Elements

The scene is composed as a small residential plot seen from the street, so that the two required houses read as a place rather than as two isolated boxes.

The building is a five-storey block of $15 \times 18 \times 11$ world units. Its facade is rendered with a custom shader that samples a brick or concrete atlas together with a window mask; the mask drives both the specular response, so glass reads as shinier than masonry, and the emissive window glow that appears after dark. A pseudo-random hash decides which windows are occupied and a slow sine term gives each lit window a subtle flicker. The block carries a plinth, a roof slab, a parapet wall, a stair head-room, a water tank on legs, four air-conditioner outdoor units, an antenna mast, an entrance canopy, a shopfront glass strip, a double door and three entrance steps.

The cottage is a $10 \times 4.6 \times 8.4$ unit gabled house rotated slightly off-axis so that two of its faces are visible from the default camera. It has two gable ends, a stone base course, two sloped roof planes meeting at a ridge cap, a brick chimney, a porch with posts, railings, a canopy and steps, a panelled wooden door, five framed windows with sills and a low garden fence. A GPU particle system of 240 points rises from the chimney as smoke.

The environment places the houses in context: a sky dome of radius 560, a 260×260 grass ground plane, an asphalt road along the front of the plot, cobblestone paths connecting the road to both entrances, soft contact-shadow decals under the props, eight trees and eight bushes that sway in the wind, flower patches, a street lamp with a glowing bulb, a

bench, boundary fencing, and distance fog whose colour is interpolated to match the sky.

Texture choices follow the material each surface is meant to represent: red brick and grey concrete atlases for the building facade, wooden planks, stone masonry, white plaster and beige brick for the cottage walls, clay tiles for the cottage roof and shingle for the flat roof structures, wood for the porch, the fence, the door and the bench, bark for the tree trunks, leaves for the canopies, grass for the ground and cobblestone for the paths. Every colour map is paired with a normal map so that the brick joints, the plank seams and the stone edges catch the moving light.

Colour and light carry the mood. During the day the sky runs from a deep blue zenith to a pale horizon and the sun is warm white; at night the dome turns near-black, stars appear with a per-star twinkle, the sun is replaced by a cool blue moon, the street lamp and the building windows switch on, and the orbiting light becomes the dominant warm accent. Shadows are rendered with 2048×2048 PCF soft shadow maps from both the sun and the orbiting point light.

The on-screen HUD is deliberately understated: dark translucent glass cards in the four corners of the screen with a single blue accent, showing the project identity, the live scene state, a short key cheat sheet and six action buttons. Pressing H hides it completely, and it is hidden automatically for one frame whenever a snapshot is taken so that the saved image contains only the 3D scene.

3.2 Snapshots

The five snapshots below were captured directly from the running project using the built-in snapshot function (camera preset keys 1 to 5, then the P key).

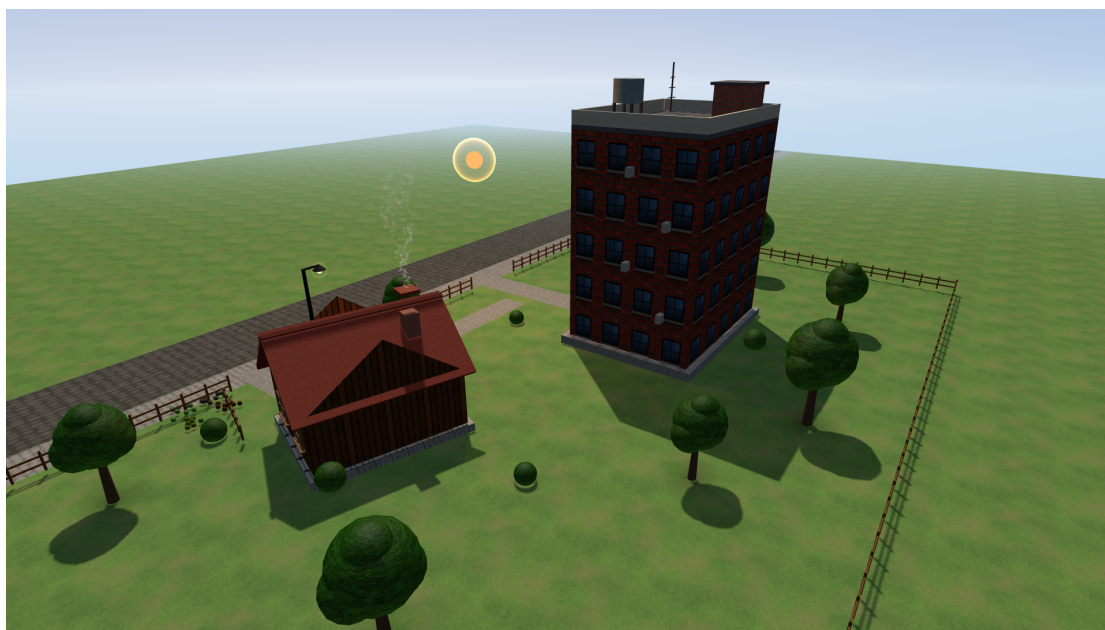


Figure 1: Front view of the plot showing the five-storey building and the gabled cottage, with the orbiting light and its circular path visible above the building.



Figure 2: Close-up of the cottage showing the wall texture with its normal map, the porch, the framed windows and the chimney smoke particles. Clicking the cottage cross-fades this texture to the next material.



Figure 3: Close-up of the building facade rendered with the custom facade shader, showing the window mask, the specular response of the glass and the roof-top water tank and AC units.

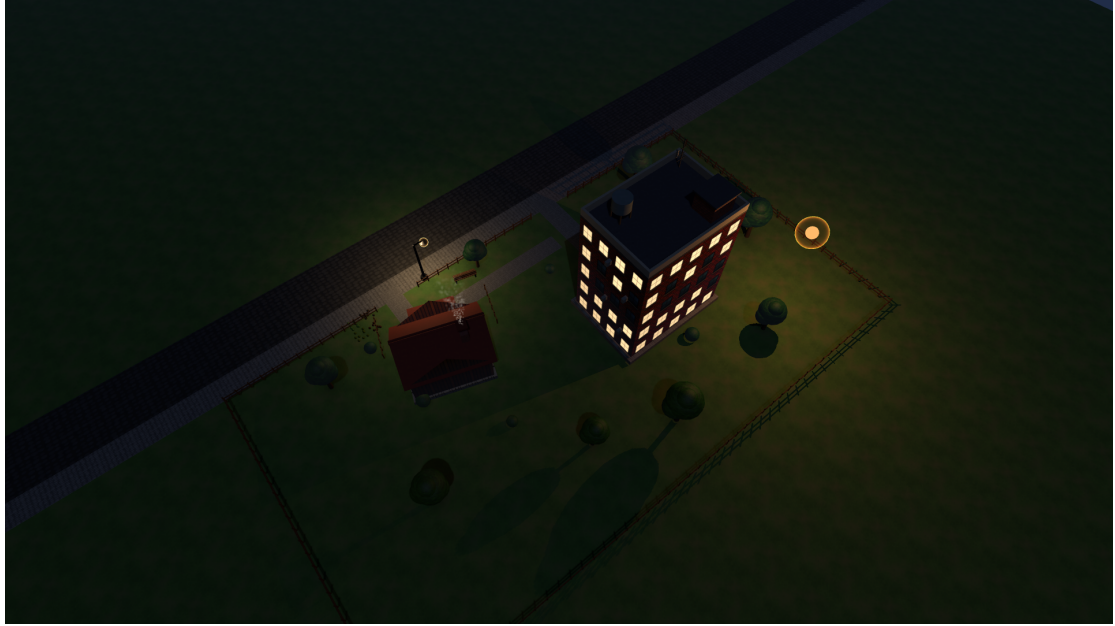


Figure 4: Aerial overview of the whole plot: road, cobblestone paths, trees, boundary fence, and the circular path of the animated light around the building.

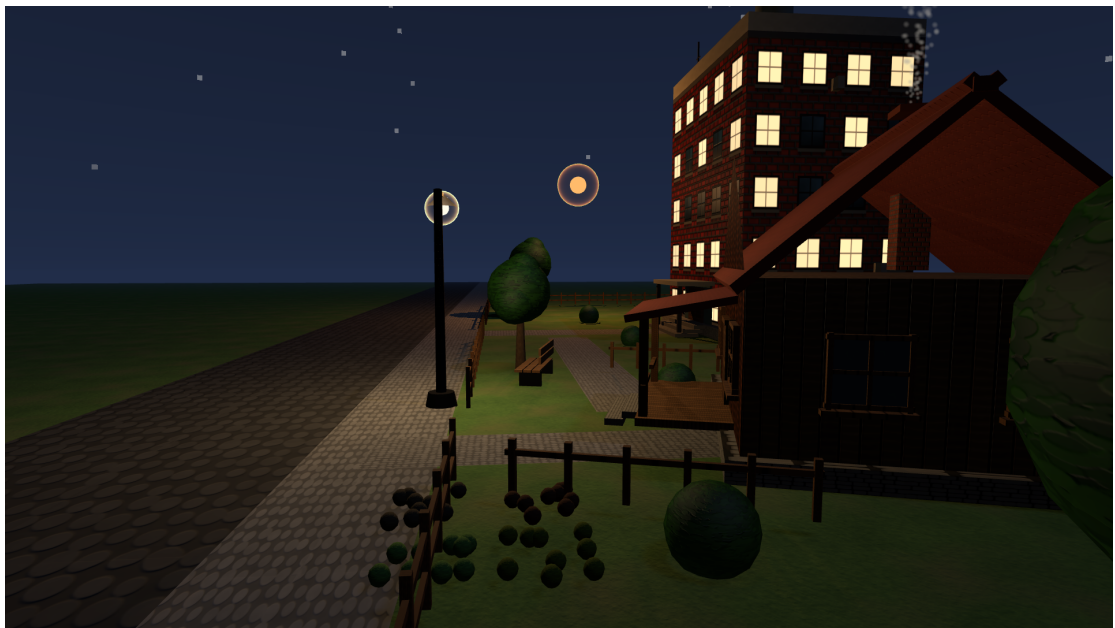


Figure 5: Night view from street level: emissive building windows, the street lamp, the star field generated in the sky shader, and the warm orbiting light sweeping across the facade.

4. Code Implementation

4.1 Software Platform

The tools and technologies used in this project are listed below.

- **Language:** JavaScript (ES2022 modules) and GLSL ES 1.0 for the shaders.

- **Graphics library:** Three.js r169, running on the WebGL 2 renderer.
- **Markup and styling:** HTML5 and CSS3 for the page shell, the HUD and the help overlay.
- **Camera controls:** written from scratch; the `OrbitControls` add-on is deliberately not used.
- **Texture generation:** Python 3 with Pillow and NumPy (`tools/gen_textures.py`).
- **Editor:** Visual Studio Code. **Report:** prepared on the supplied template.
- **Runtime:** Chrome, Edge or Firefox served from a local HTTP server.
- **Build step:** none. Native ES modules are used, so there is no bundler and no `npm install`.

Because browsers refuse to load ES modules over the `file://` protocol, the project is served by a small local HTTP server during the demonstration. Any static server is sufficient: the Live Server extension of Visual Studio Code, `python -m http.server 8204` run inside the project folder, or `node server.js`, which is included in the project folder. Three.js itself is stored in `vendor/three.module.js`, so the project runs completely offline.

4.2 Custom Shaders

Six shader programs were written for this project. All of them are `THREE.ShaderMaterial` instances and none of them use the Three.js built-in material chunks. They share a common GLSL prelude containing the sRGB conversion helpers, a hash function, a normal-mapping helper and the Blinn-Phong term, together with a common uniform block describing the four lights and the fog.

1. **Sky dome.** The vertex stage passes the world-space direction of each vertex to the fragment stage. The fragment stage builds a two-colour vertical gradient whose day and night palettes are blended by the `uNight` uniform, adds a warm scattering band near the horizon, draws the sun or moon disk with a bloom term based on `pow(dot(viewDir, sunDir), 120)`, and finally adds a hashed star field that twinkles and only appears at night.
2. **Building facade.** The vertex stage computes the tiled UV, the world position and the world normal. The fragment stage samples two facade atlases and cross-fades between them, uses a window mask texture to raise the specular strength and shininess on glass, evaluates the sun and both point lights, and then adds per-window emissive light at night with a randomised occupancy and a subtle flicker.
3. **Cottage walls.** The same vertex program is reused. The fragment stage holds two colour maps and two normal maps and mixes both pairs with the `uMix` uniform, which is animated whenever the user clicks the cottage, then adds a Fresnel rim highlight, $(1 - N \cdot V)^{2.5}$, scaled by the `uHighlight` hover uniform.
4. **Foliage.** The vertex stage displaces each vertex with two out-of-phase sine waves whose amplitude grows with height, so the canopy bends instead of sliding rigidly; the wind strength is a uniform the user can cycle. The fragment stage adds a cheap back-lit translucency term and blends the leaf colour towards an autumn amber using the `uSeason` uniform.
5. **Chimney smoke.** GPU point sprites. Each particle carries a random seed and a speed attribute, loops through its own lifetime with `fract(uTime * speed + seed)`, rises and spreads as it ages, and receives a perspective-correct `gl_PointSize`. The

fragment stage draws a soft round puff whose opacity falls off over its life and discards nearly transparent fragments.

6. **Fresnel glow.** An additive shell used for the orbiting light orb and the street-lamp bulb; brightness follows a Fresnel term so the sphere glows at its silhouette rather than at its centre.

The lighting model implemented inside these shaders consists of a hemisphere-style ambient term weighted by the y component of the surface normal, a directional sun term, and two point-light terms with a clamped falloff, $\text{att} = (1 - d/\text{range})^2$, so that no light can leak across the whole plot. Linear fog is applied last, immediately before the final sRGB encode.

4.3 Project Feature Table

The table below lists the six mandatory requirements together with the additional features implemented in this project.

#	Features	Status
1	Custom shaders — six hand-written GLSL programs: sky dome, building facade, cottage walls, foliage wind, chimney smoke and Fresnel glow	Implemented
2	Implementation of lighting — hemisphere ambient, directional sun and moon, an orbiting point light and a street lamp, with PCF soft shadow maps and a hand-written Blinn-Phong model in GLSL	Implemented
3	Perspective projection — PerspectiveCamera at 55° , near 0.1, far 1200, with a runtime-adjustable field of view	Implemented
4	Texture for each object — 14 procedurally generated tileable sets, 31 PNG files including normal maps; every mesh in the scene is textured	Implemented
5	Animation — the light position rotates around the building (required), plus a day-night cycle, wind on the foliage, chimney smoke and texture cross-fade	Implemented
6	Mouse and keyboard interaction — a raycaster click on the cottage changes its texture; the keyboard moves the camera around the houses	Implemented
7	Additional — day-night cycle with emissive building windows, a twinkling star field and a working street lamp	Implemented
8	Additional — five camera presets, an automatic orbit tour, an in-app PNG snapshot export and an on-screen help overlay	Implemented

Table 01: Project Feature Table

5. Contribution

Jakaria Mahmud (20220104141): the building model and its facade shader; the lighting system including the required orbiting-light animation, the shadow-map configuration and the day-night cycle; the procedural texture generation script and the resulting 31 texture files; and the written project report.

Md. Redwan Hossen (20220104147): the cottage model and its texture cross-fade shader; the mouse raycasting, hover feedback and click handling; the custom keyboard

and mouse camera rig; the environment, props and sky shader; the HUD and help overlay; and the project setup, local server and documentation.

Both members jointly planned the scene layout, reviewed each other's shaders, calibrated the lighting so that the custom shaders and the built-in materials match, and tested the project on a machine without internet access to confirm that the offline path works before the demonstration.